RTLWire Studio

功能、使用流程与实现原理

让连接可见，
让判定可复查。

ELAB 接线检查
FSDB 隔离与掉电检查
Linux 原生 Tk 桌面

VERSION 0.5.0



三种工作方式，一套证据链

从“是否接到端口”，到“经过哪些实例”，再到“掉电行为是否符合规则”。

01

连接检查 · reach

父层信号是否连到目标 module 实例的输入口或输出口?

ELAB → 端口命中与位覆盖

02

连接顺序 · trace

信号从起点向下游，跨越哪些层次接口和 module 实例?

ELAB → 分支路径与停止原因

03

掉电波形检查

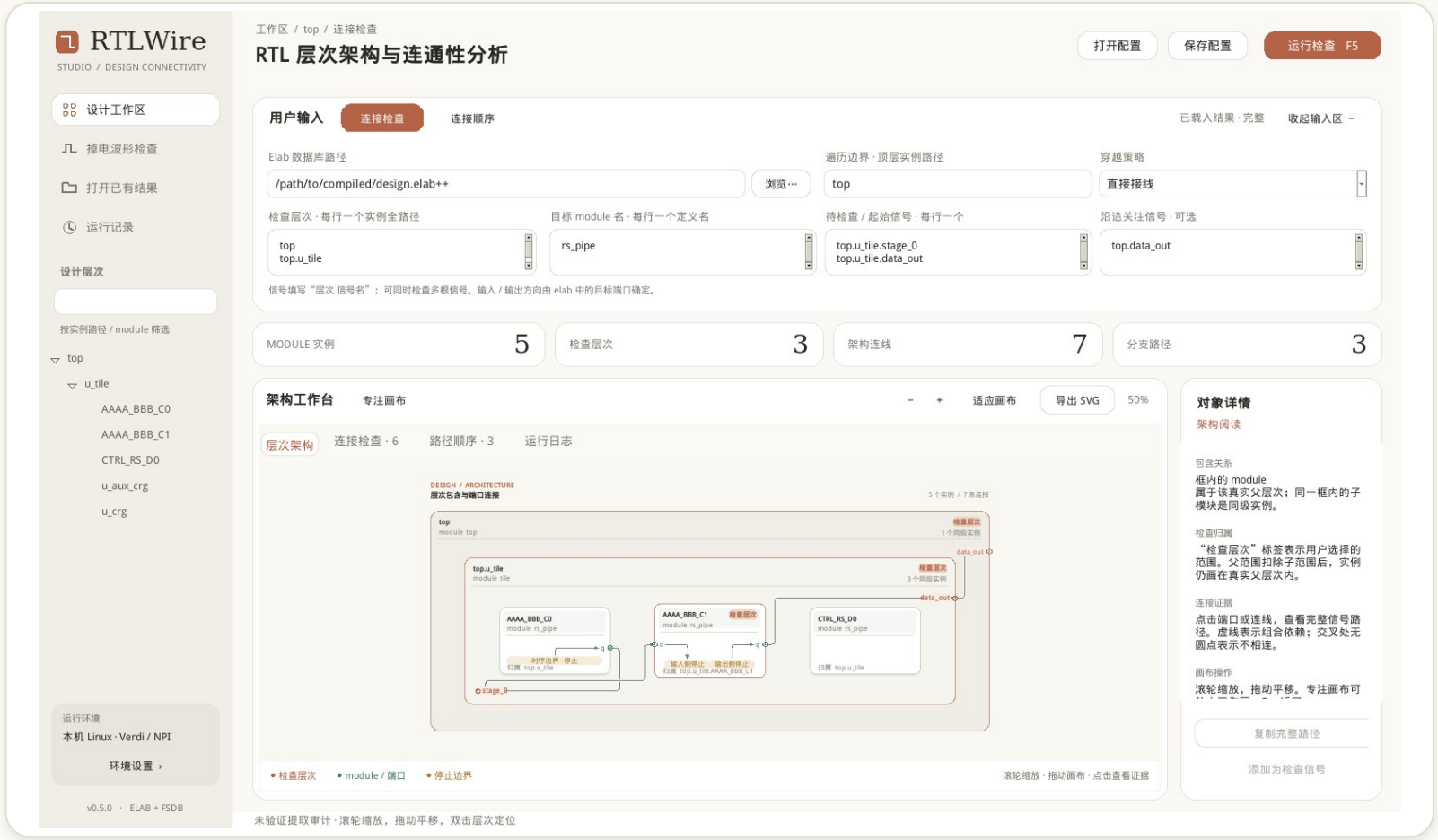
隔离、掉电与恢复期间，输出是否符合该层 RPT 分组规则?

FSDB + RPT → 逐口判定

共同输出： JSON / CSV / SVG + 输入快照、错误位置与检查证据

输入层次、 module 与信号，直接运行

输入区默认展开；打开历史结果后仍然可以继续编辑检查配置。



1 已有 elab 路径

指定编译库和遍历边界；
不提供 RTL filelist 。

2 三类检查对象

检查层次、 module 定义名、
待检查或起始信号。

3 结果与证据联动

选中端口或连线查看详情；
可切换路径、日志和 SVG 。

两根父层信号，分别检查同一实例的输入与输出

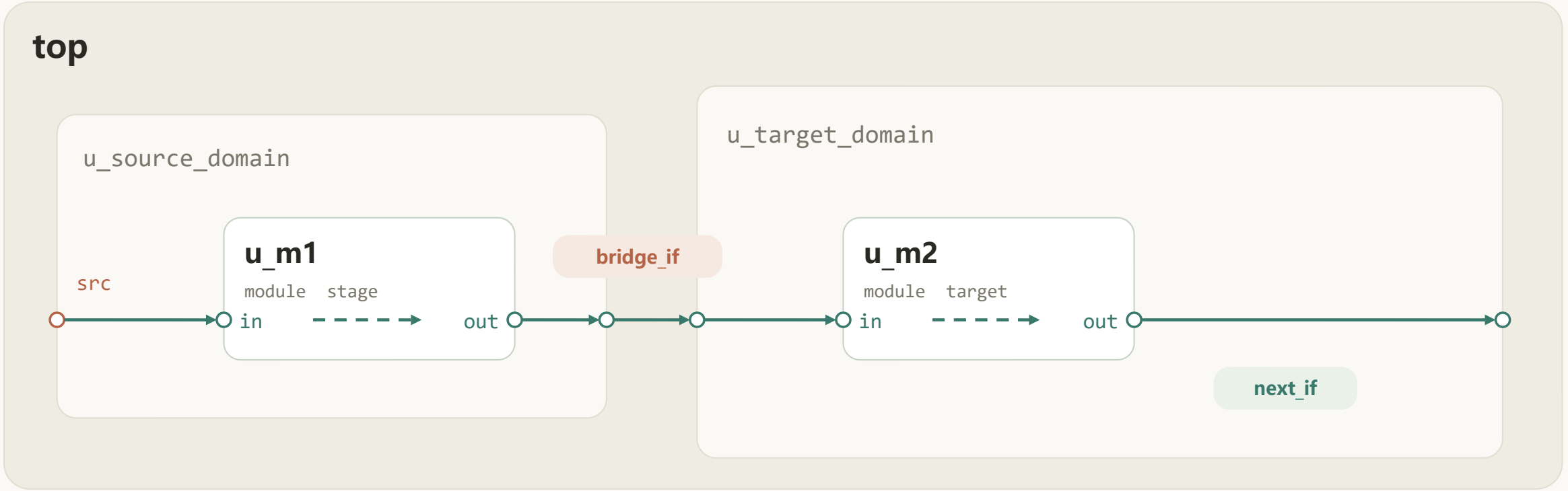
目标 module 可以位于任意深度；两根信号分别生成检查结论。



两个独立问题： data_in → d ； q → data_out 。是否存在 d → q 的内部依赖，需要另有 NPI 证据。

跨接口、跨层次，找出连接顺序

关注信号用于保留关键节点，不是限制遍历的白名单。以下为机制示意。




```
src → u_m1.in ⇢ u_m1.out → bridge_if → u_m2.in ⇢ u_m2.out → next_if
```

实线：实际连接；虚线：已经提取到的组合依赖。遇到时序单元或不支持的映射会明确停止。

检查归属与真实包含关系，分别保存

父范围扣除另行选择的子树；图上仍保留真实的父子层次。



1 最近的已选祖先

u_c0 及其后代归属于 u_c0；u_c1 归属于 top。

2 父范围扣除子范围

top 的统计不重复计入 u_c0 整棵已选子树。

3 显示不重新挂接

u_c0 与 u_c1 仍放在真实父层次 u_tile 内。

从编译库到检查结论：可追溯的数据通路

提取、分析、展示各有独立职责；原始连接证据随结果保留。

1
已编译 ELAB
用户已有编译产物

→

2
Tcl NPI 提取
实例 / 端口 / Driver / Load

→

3
标准化原始图
节点、边、位范围与证据

→

4
检查与路径分析
归属 / 可达性 / 分支

→

5
GUI 与导出
架构图 / JSON / CSV / SVG

事实层

原始记录保留对象 ID 、完整路径、真实层次、方向、位范围和原始边索引。

展示层

合并端口绑定、压缩中间 net 的显示；点击图形仍能回到原始连接证据。

NPI 通过对象句柄读取连接事实

加载已有库、建立对象清单，再按方向遍历连接关系。

1 **加载** `debImport -elab`

仅加载已有的编译数据库

2 **枚举** `npi_nl_iterate / npi_nl_scan`

实例、端口、 net 与定义名

3 **定位** `npi_nl_sig_handle_by_name`

将用户信号解析为对象 / 位范围

4 **追踪** `npiNlLoad / npiNlDriver`

读取负载与驱动，保留边证据

位宽、方向与真实层次均来自 NPI；无法证明的映射明确停止。

reach 做可达性检查，trace 列举下游分支

同一张原始有向图，在两种模式下采用不同的查询方式。

reach：按源位执行 BFS

1 建立双向邻接索引用于连通性搜索

2 匹配目标实例的端口与源位覆盖

3 每个命中端口保留一条见证路径

trace：有向图路径枚举

1 仅沿下游邻接表展开路径

2 保留分支、关注点及停止原因

3 发现循环或达到资源限制时终止分支

CONNECTED　PARTIAL_BITS　NOT_CONNECTED　UNKNOWN

“未连通”只在提取完整、边界与策略明确时成立。

只读编译库：入口、运行时与审计共同约束

这些措施约束本工具的加载流程，不等同于任意程序的通用安全沙箱。

01 入口限制

只接受已有 elab；不接收 RTL filelist、宏、include 或参数覆盖。

02 只读挂载

在独立 mount namespace 内，把输入库与解析到的依赖目录只读挂载。

03 源码访问保护

LD_PRELOAD 保护库拒绝常见 RTL 源文件后缀的打开请求。

04 访问与完整性审计

strace 记录文件 / 进程访问；前后 SHA256 比较确认输入未改动。

架构图不是平铺节点图，而是真实层次的投影

布局与绘制共用图形元素，界面选中可关联到连接见证。



包含树

从真实 parent 构建嵌套容器

同级布局

按可见连接分组和排序

端口与布线

端口放在边界，正交路径避开其他模块

共享图元

Tk 与 SVG 共用矩形、文字、端口和连线

两个层次，独立配置控制信号与 RPT

枚举输出口、检查分组规则，并在同一界面查看周期波形与首次异常。



1 每层独立输入

层次、 iso_en 、power_down 、RPT 文件和三类关键词。

2 自动读取输出口

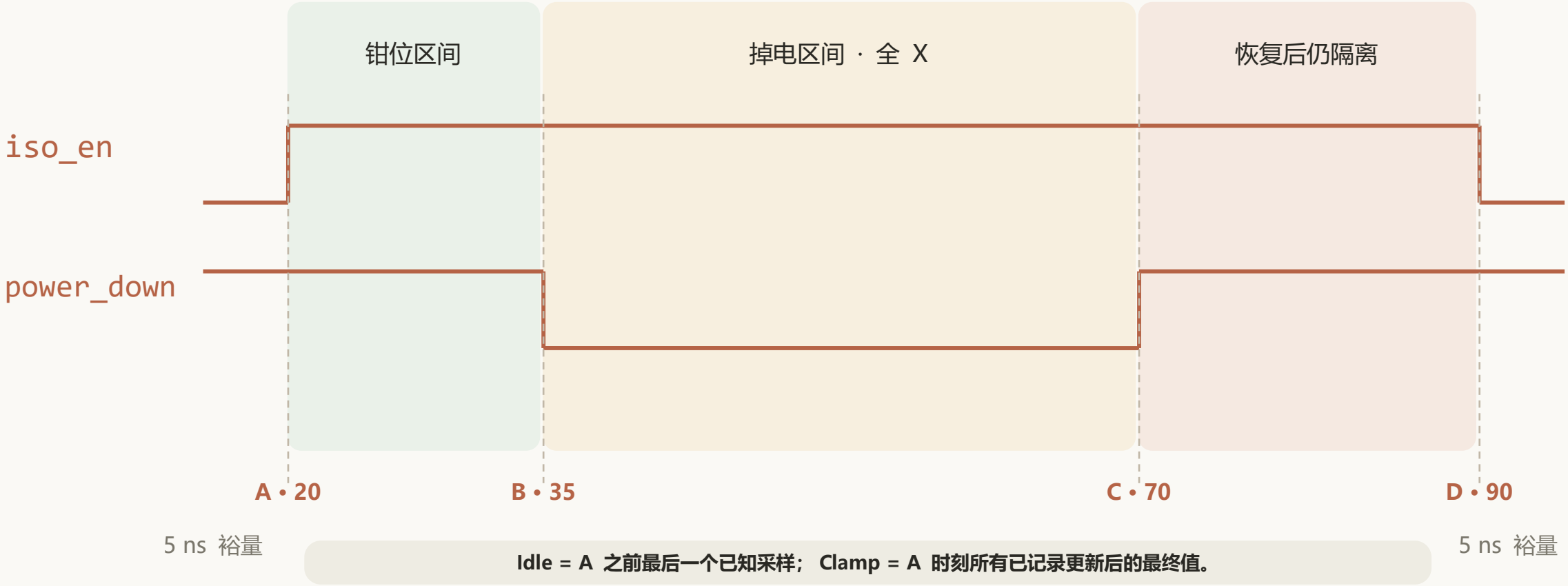
只使用 FSDB 已记录且具有方向信息的本层 output 。

3 定位具体问题

选择输出口，在完整周期中查看首个异常及实际 / 期望值。

四个边沿定义完整周期，前后各留 5 ns

示例： A=20 ns， B=35 ns， C=70 ns， D=90 ns；显示窗口为 15–95 ns。



Idle = A 之前最后一个已知采样； Clamp = A 时刻所有已记录更新后的最终值。

组名由用户定义，关键词负责分类

组名包含关键词，区分大小写；每组只能匹配一种类型。左侧为 RPT 节选。

```
ALPHA_KEEP1  { one_out, one_bus }

ALPHA_KEEP0  { zero_out, zero_bus }

ALPHA_BYPASS { free_out, free_bus }
```

用户输入关键词	识别类型
KEEP1	钳 1
KEEP0	钳 0
BYPASS	不钳位



各层可使用不同文件与关键词。支持中文 / 英文逗号、空格、换行、注释及位 / 片选。

一根总线可以按位混合分组

检查规则落实到 FSDB 声明的位号，而不是仅按信号名字归类。

```
KEEP1  { mixed_bus[3] }
KEEP0  { mixed_bus[2] }
BYPASS { mixed_bus[1:0] }
```

bit 3	bit 2	bit 1	bit 0
1	0	—	—
钳 1	钳 0	不钳位	不钳位

期望掩码

10--

只比较需要钳位的位

钳位相关规则：只比较 bit 3 和 bit 2 。
掉电全 X 规则：检查 bit 3…0 全部位。

五项规则覆盖隔离、掉电与恢复

“不钳位”跳过钳位相关规则，掉电期间仍必须全 X。

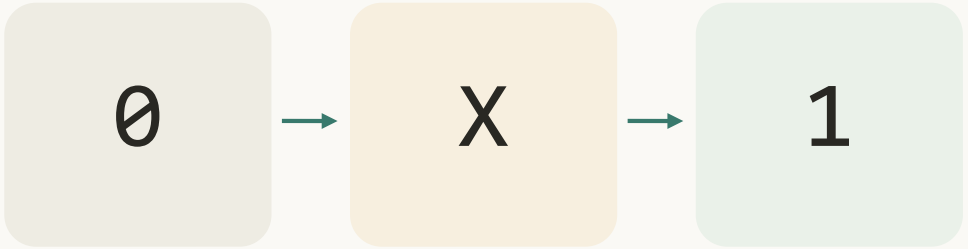
检查项	钳 1	钳 0	不钳位
A 前 idle = A 最终值	检查	检查	SKIP
[A,B) 符合期望值	全 1	全 0	SKIP
[A,B) 保持稳定	检查	检查	SKIP
[B,C) 掉电值	全 X	全 X	全 X
[C,D) 恢复后的隔离值	全 1	全 0	SKIP

由于还检查 idle 与钳位值相等：钳 1 组通过时，相关位的 idle 也须为 1；钳 0 同理。

同一时间戳取最终值，数据不足明确报未知

这是已确认的判定口径；短暂的已记录 delta 更新作为提示。

同一时刻 20 ns 的已记录更新



最终用于判定： 1

delta 作为提示保留。 FSDB 未保存的瞬态变化，不能由工具事后重建。

PASS 规则已经验证通过

FAIL 有明确违例和首个失败位置

UNKNOWN 边沿、方向、波形或分组覆盖不足

SKIP 该规则对不钳位的信号位不适用

C++ Reader 提取事件，Python 执行检查

使用本机安装的 Reader SDK，输出结构化记录，不把 FSDB 转成推测的设计端口表。

1	**读取树与元数据**	`ffrReadScopeVarTree2` scope / module / direction / 位号
2	**选择记录对象**	控制信号 + 直接 output 严格匹配层次与 output 方向
3	**遍历四态事件**	`ffrCreateVCTraverseHandle` 64 位时间戳与 0 / 1 / X / Z
4	**标准化和判定**	JSONL → Series → 周期状态机 时间单位换算、掩码规则、首个异常

界面、任务进程和分析内核分层

耗时的 EDA / FSDB 读取在后台运行， GUI 通过事件队列接收进度与结果。

Tk / ttk 视图

gui.py / theme.py
wave_gui.py

输入、表格、选择联动
缩放、平移、 SVG 导出

任务与 CLI

jobs.py
cli.py / wave_cli.py

独立运行目录
进度事件、取消、审计

提取与规则内核

NPI / FSDB Reader
core.py / wave_core.py

原始事实、图算法
分组、时序与状态聚合

共享绘图层： architecture.py / wave_plot.py → Tk Canvas + SVG

用正常、故障和缺失数据，验证检查器行为

数字来自 0.5.0 开发验收记录；它们代表已验证案例，不等于全芯片覆盖率。

62 **单元测试**

9 **实际 FSDB GUI 流程**

6 **界面与布局检查**

独立测试波形	通过	失败	未知
三类分组 + 混合总线	14	0	0
注入 6 类输出违例	8	6	0
重复 / 未分组输出	12	0	2

额外验证：普通账号读取只读 FSDB / RPT 、打包后启动、旧报告兼容、 SVG 导出。

Linux 本机部署，支持范围清楚可见

已验证环境： CentOS 7 · Python 3.6.8 · Tk 8.5 · Verdi O-2018.09-SP2 。

运行条件

GUI： Python 标准库 + Tk + 中文字体

ELAB： 本机 Verdi / NPI 与许可证

FSDB： 本机 Reader SDK 、编译器与 strace

当前支持边界

时序单元明确停止，不凭空跨越 D → Q

复杂或不确定的组合映射报告停止 / 未知

FSDB 只覆盖已 dump 且具方向的直接 output

NPI 只读挂载需要相应权限； FSDB 分支可由普通账号运行。不同 SDK 版本需确认兼容性。

配置 → 检查 → 定位 → 保存

每一次运行产生独立目录，便于复查配置、结论与证据。

1

配置

ELAB 或 FSDB
层次 / 信号 / RPT

2

检查

后台执行
查看进度与日志

3

定位

架构对象或波形
首个异常及完整路径

4

保存

JSON / CSV / SVG
配置与结果目录

```
sh rtlwire-studio --result validation/reach_dual_io
sh rtlwire-studio --result validation/wave_groups_faults
```

从功能定位到实现文件

用于代码评审、维护和后续扩展。图形与核心规则均可独立阅读。

功能	主要实现	关键职责
ELAB 提取	npi/extract.tcl	枚举实例 / 端口，追踪 Driver / Load
接线分析	rtlwire/core.py	最近祖先归属、 BFS 、路径枚举
架构绘图	rtlwire/architecture.py	真实包含、布局、正交布线与 SVG
FSDB 读取	wave/fsdb_reader.cpp	方向与位号、原生 tick 、四态事件
RPT 与规则	wave_groups.py / wave_core.py	按位归组、周期状态机和区间检查
界面与任务	gui.py / wave_gui.py / jobs.py	用户输入、异步进度、选择联动
只读与审计	cli.py / wave_cli.py / source_guard.c	输入保护、日志、 SHA256 与资源限制

连接有路径，判定有依据。

以编译库与波形记录为事实来源，把检查过程做成可复查的工程工具。

两类接线检查
三类输出分组
一个本地桌面工作区

交付内容

源码与可运行示例
PPT / PDF 与实际界面截图
配置说明与核心测试

github.com/ysyx-22040210-yudian/rtlwire-studio